18.11 Security Process Improvement and Maturity Models

A number of process improvement and maturity models are available for software development in general, such as the Capability Maturity Model Integration (CMMI).[13] For cyber security maturity, the CMMI Institute offers a newer product, the Cyber Capability Maturity Management platform.[14] OWASP offers the Software Assurance Maturity Model (SAMM).[15] SAMM is an open framework to help organizations formulate and implement a strategy for software security that is tailored to the specific risks facing the organization.

A comprehensive discussion of these models is beyond the scope of this book. To provide a simple overview, consider the overall objective of SAMM:

- Evaluate an organization's existing software security practices.
- Build a balanced software security assurance program in well-defined iterations.
- Demonstrate concrete improvements to a security assurance program.
- Define and measure security-related activities throughout an organization.

Perhaps the most well-known maturity model that is specifically for software security is the Building Security in Maturity Model (BSIMM). BSIMM has periodic releases, typically every year or two. The BSIMM model and its recent summarized assessment results can be downloaded from the BSIMM website.[16] According to the BSIMM developers, BSIMM is meant to be used by those who create and execute a software security initiative.

All the maturity models mentioned here (and others) have benefits, and the essential elements of the models are freely available. However, sometimes the assessment is done by external entities. It's possible to do an internal self-assessment and define the associated improvement program, but it requires dedicated resources and effort. Alternatively, some of these organizations offer assessment programs, thus providing an external view of the strengths and areas for improvement within a software organization.

18.12 Summary

All software engineers should have an understanding of what it takes to develop secure software. The steps that are needed to improve the security of your software products are relevant in each activity in the software process, regardless of which process model is used.

Although there are still many open questions, and technologies that need additional research, there are many resources available today to assist with this challenge. For every activity that normally takes place in the software process, try to incorporate

13 See https://cmmiinstitute.com/.
14 See https://cmmiinstitute.com/products/cybermaturity.
15 See https://www.owasp.org/index.php/OWASP_SAMM_Project.
16 See https://www.bsimm.com/.

security aspects. Models such as Microsoft SDL and the SQUARE model can be assessed to determine which steps you can incorporate into your development process.

Add security to a risk analysis activity, especially using detailed guidance available from NIST. Given the number of secure coding standards already available, it is certainly possible for anyone to learn how to code securely. Inspect your code for remaining vulnerabilities. Learn how to identify security holes and develop and prioritize mitigation strategies. Perform static analysis testing on your code. Visit the OWASP and BSIMM websites, among others, to learn about maturity in software security engineering.

As software becomes ever more ubiquitous, the number of vulnerabilities and successful hacks grow as well. It will take all our efforts to stem the tide, but many of the tools already exist to tackle this problem. The consequences of failing to address software security are high, and the benefits of developing secure software are huge.

PROBLEMS AND POINTS TO PONDER

18.1. What is the most important thing that a software team can do to improve software security?

18.2. If you were recommending one activity for your organization to improve software security, what would it be? If you were recommending multiple activities, what are they, and what would be the priorities, considering that it's not likely that all of them will be implemented at once?

18.3. How could you incorporate software security into your existing process model or into a new process model?

18.4. Sit down with a colleague and identify security risks on a software project that is in development. Come up with mitigation strategies and prioritize them.

18.5. Are you collecting measurement data that could be used or repurposed to help measure software security? If not, is there data that could easily be collected for this purpose?

18.6. Use the Internet to find out the details needed to create a phishing attack pattern.

18.7. Explain some of the problems that might be encountered if you wait until after the system is completed to address security risks.

18.8. Use the Internet to determine the average cost to the consumer of a single incidence of identity theft.

18.9. Consider a MobileApp that you make use of on your personal phone. List three to five security risks that developers should consider when developing apps like this one.

18.10. Determine the security requirements of a bill-paying wallet-type MobileApp.

Design element: Quick Look icon magnifying glass: © Roger Pressman

CHAPTER

19 SOFTWARE TESTING—COMPONENT LEVEL

Software component testing incorporates a strategy that describes the steps to be conducted as part of testing, when these steps are planned and then undertaken, and how much effort, time, and resources will be required. Within the testing strategy, software component testing implements a collection of component testing tactics that address test planning, test-case design, test execution, and resultant data collection and evaluation. Both component testing strategy and tactics are considered in this chapter.

KEY CONCEPTS

basis path testing 384
black-box testing 388
boundary value analysis 389
class testing 390
control structure testing 386
cyclomatic complexity 385
debugging 373
equivalence partitioning 389
independent test group 375
integration testing 375
interface testing 388
object-oriented testing 390
scaffolding 379
system testing 376
testing methods 373
testing strategies 373
unit testing 376
validation testing 376
verification 373
white-box testing 383

QUICK LOOK

What is it? Software is tested to uncover errors that were made inadvertently as it was designed and constructed. A software component testing strategy considers testing of individual components and integrating them into a working system.

Who does it? A software component testing strategy is developed by the project manager, software engineers, and testing specialists.

Why is it important? Testing often accounts for more project effort than any other software engineering action. If it is conducted haphazardly, time is wasted, unnecessary effort is expended, and even worse, errors sneak through undetected.

What are the steps? Testing begins "in the small" and progresses "to the large." By this we mean that early testing focuses on a single component or on a small group of related components and applies tests to uncover errors in the data and processing logic that have been encapsulated by the component(s). After components are tested, they must be integrated until the complete system is constructed.

What is the work product? A *test specification* documents the software team's approach to testing by defining a plan that describes an overall strategy and a procedure that defines specific testing steps and the types of test cases that will be conducted.

How do I ensure that I've done it right? An effective test plan and procedure will lead to the orderly construction of the software and the discovery of errors at each stage in the construction process.

To be effective, a component testing strategy should be flexible enough to promote a customized testing approach but rigid enough to encourage reasonable planning and management tracking as the project progresses. Component testing remains a responsibility of individual software engineers. Who does the testing, how engineers communicate their results with one another, and when testing is done is determined by the software integration approach and design philosophy adopted by the development team.

These "approaches and philosophies" are what we call *strategy and tactics*—topics to be discussed in this chapter. In Chapter 20 we discuss the integration testing techniques that often end up defining the team development strategy.

19.1 A Strategic Approach to Software Testing

Testing is a set of activities that can be planned in advance and conducted systematically. For this reason, a template for software testing—a set of steps into which we can place specific test-case design techniques and testing methods—should be defined for the software process.

A number of software testing strategies have been proposed in the literature [Jan16] [Dak14] [Gut15]. All provide you with a template for testing, and all have the following generic characteristics:

- To perform effective testing, you should conduct technical reviews (Chapter 16). By doing this, many errors will be eliminated before testing commences.
- Testing begins at the component level and works "outward" toward the integration of the entire computer-based system.
- Different testing techniques are appropriate for different software engineering approaches and at different points in time.
- Testing is conducted by the developer of the software and (for large projects) an independent test group.
- Testing and debugging are different activities, but debugging must be accommodated in any testing strategy.

A strategy for software testing incorporates a set of tactics that accommodate the low-level tests necessary to verify that a small source code segment has been correctly implemented as well as high-level tests that validate major system functions against customer requirements. A strategy should provide guidance for the practitioner and a set of milestones for the manager. Because the steps of the test strategy occur at a time when deadline pressure begins to rise, progress must be measurable and problems should surface as early as possible.

19.1.1 Verification and Validation

Software testing is one element of a broader topic that is often referred to as verification and validation (V&V). *Verification* refers to the set of tasks that ensure that software correctly implements a specific function. *Validation* refers to a different set

of tasks that ensure that the software that has been built is traceable to customer requirements. Boehm [Boe81] states this another way:

> Verification: "Are we building the product right?"
> Validation: "Are we building the right product?"

The definition of V&V encompasses many software quality assurance activities (Chapter 19).[1]

Verification and validation include a wide array of SQA activities: technical reviews, quality and configuration audits, performance monitoring, simulation, feasibility study, documentation review, database review, algorithm analysis, development testing, usability testing, qualification testing, acceptance testing, and installation testing. Although testing plays an extremely important role in V&V, many other activities are also necessary.

Testing does provide the last bastion from which quality can be assessed and, more pragmatically, errors can be uncovered. But testing should not be viewed as a safety net. As they say, "You can't test in quality. If it's not there before you begin testing, it won't be there when you're finished testing." Quality is incorporated into software throughout the process of software engineering, and testing cannot be applied as a fix at the end of the process. Proper application of methods and tools, effective technical reviews, and solid management and measurement all lead to quality that is confirmed during testing.

19.1.2 Organizing for Software Testing

For every software project, there is an inherent conflict of interest that occurs as testing begins. The people who have built the software are now asked to test the software. This seems harmless in itself; after all, who knows the program better than its developers? Unfortunately, these same developers have a vested interest in demonstrating that the program is error-free, that it works according to customer requirements, and that it will be completed on schedule and within budget. Each of these interests mitigates against thorough testing.

From a psychological point of view, software analysis and design (along with coding) are constructive tasks. The software engineer analyzes, models, and then creates a computer program and its documentation. Like any builder, the software engineer is proud of the edifice that has been built and looks askance at anyone who attempts to tear it down. When testing commences, there is a subtle, yet definite, attempt to "break" the thing that the software engineer has built. From the point of view of the builder, testing can be considered to be (psychologically) destructive. So the builder treads lightly, designing and executing tests that will demonstrate that the program works, rather than to uncover errors. Unfortunately, errors will be nevertheless present. And, if the software engineer doesn't find them, the customer will!

---

1 It should be noted that there is a strong divergence of opinion about what types of testing constitute "validation." Some people believe that *all* testing is verification and that validation is conducted when requirements are reviewed and approved, and later, by the user when the system is operational. Other people view unit and integration testing (Chapters 19 and 20) as verification and higher-order testing (Chapter 21) as validation.

There are often a number of misconceptions that you might infer from the preceding discussion: (1) that the developer of software should do no testing at all, (2) that the software should be "tossed over the wall" to strangers who will test it mercilessly, and (3) that testers get involved with the project only when the testing steps are about to begin. Each of these statements is incorrect.

The software developer is always responsible for testing the individual units (components) of the program, ensuring that each performs the function or exhibits the behavior for which it was designed. In many cases, the developer also conducts *integration testing*—a testing step that leads to the construction (and test) of the complete software architecture. Only after the software architecture is complete does an independent test group become involved.

The role of an *independent test group* (ITG) is to remove the inherent problems associated with letting the builder test the thing that has been built. Independent testing removes the conflict of interest that may otherwise be present. After all, ITG personnel are paid to find errors.

However, you don't turn the program over to ITG and walk away. The developer and the ITG work closely throughout a software project to ensure that thorough tests will be conducted. While testing is conducted, the developer must be available to correct errors that are uncovered.

The ITG is part of the software development project team in the sense that it becomes involved during analysis and design and stays involved (planning and specifying test procedures) throughout a large project. However, in many cases the ITG reports to the software quality assurance organization, thereby achieving a degree of independence that might not be possible if it were a part of the software engineering team.

19.1.3 The Big Picture

The software process may be viewed as the spiral illustrated in Figure 19.1. Initially, system engineering defines the role of software and leads to software requirements analysis, where the information domain, function, behavior, performance,

FIGURE 19.1

Testing strategy

